

OWASP Top 10 : 2025

Comprehensive Study Notes — Week 1

Penetration Testing Internship | Zaalima Development Pvt Ltd

Reference: owasp.org/Top10/2025/

The OWASP Top 10 is the globally recognized standard for the most critical web application security risks. This document summarizes all 10 vulnerabilities from the 2025 edition — covering definitions, real-world impact, attack examples, and prevention techniques.

A01:2025 Broken Access Control

Definition	Users are able to act outside their intended permissions — accessing, modifying, or deleting data they shouldn't be allowed to.
Impact	Unauthorized data access, privilege escalation, full system compromise.
Real-World Examples	- Changing a URL parameter (e.g., /account?id=123 → /account?id=456) to view another user's data - Accessing admin pages without admin privileges - Manipulating JWT tokens to gain elevated roles
How to Prevent	- Deny access by default — whitelist what is allowed - Enforce access control checks server-side, not just client-side - Log and monitor access control failures - Invalidate session tokens after logout

A02:2025 Security Misconfiguration

Definition	Insecure default settings, incomplete configurations, open cloud storage, verbose error messages, or unnecessary features left enabled.
Impact	Attackers can exploit exposed settings to gain unauthorized access or extract sensitive information.
Real-World Examples	- Default admin credentials not changed (admin/admin) - Detailed error messages exposing stack traces to users - Unnecessary services or ports left open - Missing HTTP security headers
How to Prevent	- Remove or disable unused features, ports, and services - Change all default credentials immediately after setup - Use automated tools to verify configurations - Display generic error messages — never expose stack traces

A03:2025 Software Supply Chain Failures

Definition	Vulnerabilities introduced through third-party libraries, plugins, or dependencies that are outdated, unverified, or compromised.
Impact	A single compromised dependency can affect thousands of applications that rely on it.
Real-World Examples	- Using outdated npm/pip packages with known CVEs - Downloading plugins from untrusted sources - SolarWinds-style attack — malicious code injected into a legitimate software update
How to Prevent	- Regularly audit and update third-party dependencies - Use tools like OWASP Dependency-Check or Snyk - Only use packages from official, trusted repositories - Verify checksums and signatures of downloaded packages

A04:2025 Cryptographic Failures

Definition	Sensitive data (passwords, financial info, health records) transmitted or stored without proper encryption, or using weak/outdated algorithms.
Impact	Exposure of sensitive data leading to identity theft, fraud, or regulatory violations.
Real-World Examples	- Storing passwords in plain text or with weak MD5/SHA1 hashing - Transmitting sensitive data over HTTP instead of HTTPS - Using weak encryption keys or outdated algorithms like DES
How to Prevent	- Use strong algorithms: AES-256 for encryption, bcrypt/Argon2 for passwords - Always enforce HTTPS for data in transit - Never store sensitive data unnecessarily - Use up-to-date TLS (1.2 or 1.3) configurations

A05:2025 Injection

Definition	Malicious data is sent to an interpreter as part of a command or query — the attacker's input is executed as code.
Impact	Full database compromise, authentication bypass, remote code execution.
Real-World Examples	- SQL Injection: ' OR 1=1-- in a login field bypasses authentication - XSS (Cross-Site Scripting): <script>alert('hacked')</script> injected into a comment field - Command Injection: passing system commands through unsanitized input fields
How to Prevent	- Use parameterized queries / prepared statements for database calls - Validate and sanitize ALL user input - Use a Web Application Firewall (WAF) - Apply the principle of least privilege to database accounts

A06:2025 Insecure Design

Definition	Security flaws baked into the architecture or design of the application — not implementation bugs, but missing security controls from the start.
Impact	Fundamental weaknesses that are expensive to fix after deployment and can be exploited in multiple ways.
Real-World Examples	- No rate limiting on login — allows brute force attacks - Password reset flow that reveals whether an email exists - No separation of duties — a single user can both create and approve transactions
How to Prevent	- Apply threat modeling during the design phase - Use security design patterns and principles (e.g., least privilege, defense in depth) - Conduct design reviews with security teams before development - Create secure user stories with misuse/abuse cases

A07:2025 Authentication Failures

Definition	Weaknesses in authentication or session management that allow attackers to impersonate users or steal sessions.
Impact	Account takeover, unauthorized access to sensitive data, identity theft.
Real-World Examples	- Allowing weak passwords like '123456' or 'password' - Session tokens exposed in URLs (e.g., example.com/page?sessionid=abc123) - No lockout after multiple failed login attempts — enables brute force
How to Prevent	- Enforce multi-factor authentication (MFA) for all users - Implement account lockout after repeated failed logins - Use secure, random session IDs and invalidate them after logout - Never expose session tokens in URLs or logs

A08:2025 Software or Data Integrity Failures

Definition	Code or data pipelines that are not protected against unauthorized modification — often involving auto-updates, CI/CD pipelines, or serialized data.
Impact	Malicious code execution, data tampering, backdoor installation.
Real-World Examples	- An application auto-updates from an untrusted CDN without signature verification - Deserialization of untrusted data leading to remote code execution - Malicious code pushed through a compromised CI/CD pipeline
How to Prevent	- Verify digital signatures of downloaded software/updates - Use trusted repositories and enforce dependency integrity checks - Never deserialize data from untrusted sources

- Implement code review and approval gates in CI/CD pipelines

A09:2025 Security Logging and Alerting Failures

Definition	Insufficient logging and monitoring means attacks go undetected, and there is no ability to respond to breaches in time.
Impact	Breaches go undetected for long periods, increasing damage and recovery cost.
Real-World Examples	• Failed login attempts are not logged • No alerts triggered for suspicious activity like mass data downloads • Logs are stored locally and can be deleted by an attacker
How to Prevent	• Log all authentication events, access control failures, and input validation errors • Use a centralized, tamper-proof SIEM (Security Information and Event Management) system • Set up automated alerts for suspicious patterns • Regularly review and test your monitoring and alerting system

A10:2025 Mishandling of Exceptional Conditions

Definition	Errors, exceptions, or edge cases that are not properly handled — revealing sensitive information or causing the application to fail in an insecure state.
Impact	Information disclosure, denial of service, unexpected behavior that can be exploited.
Real-World Examples	• Unhandled exceptions revealing full stack traces to end users • Application crashes when given unexpected input types • Error messages disclosing internal server paths, database names, or version numbers
How to Prevent	• Implement global exception handlers — never show raw errors to users • Show generic error messages; log detailed errors server-side only • Test all edge cases and boundary conditions thoroughly • Fail securely — ensure errors don't leave the app in a vulnerable state

Quick Reference Summary

#	Vulnerability	Key Risk	Primary Fix
A01	Broken Access Control	Unauthorized data access, privilege escalation, full syst...	Deny access by default — whitelist what is allowed
A02	Security Misconfiguration	Attackers can exploit exposed settings to gain unauthoriz...	Remove or disable unused features, ports, and services
A03	Software Supply Chain Failures	A single compromised dependency can affect thousands of a...	Regularly audit and update third-party dependencies
A04	Cryptographic Failures	Exposure of sensitive data leading to identity theft, fra...	Use strong algorithms: AES-256 for encryption, bcrypt/Argon2 f...
A05	Injection	Full database compromise, authentication bypass, remote c...	Use parameterized queries / prepared statements for database c...
A06	Insecure Design	Fundamental weaknesses that are expensive to fix after de...	Apply threat modeling during the design phase
A07	Authentication Failures	Account takeover, unauthorized access to sensitive data, ...	Enforce multi-factor authentication (MFA) for all users
A08	Software or Data Integrity Failures	Malicious code execution, data tampering, backdoor instal...	Verify digital signatures of downloaded software/updates
A09	Security Logging and Alerting Failures	Breaches go undetected for long periods, increasing damag...	Log all authentication events, access control failures, and in...
A10	Mishandling of Exceptional Conditions	Information disclosure, denial of service, unexpected beh...	Implement global exception handlers — never show raw errors to...